SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING PROGRAMME

CSA37- SOFTWARE TESTING

1.Select any system (e.g., ATM system) and thoroughly examine its system specifications to identify and document the different software defects present.

2.Write a case study to identify and analyze the requirements of ARMS Portals, and prepare positive test scenarios as well as test cases for the application.

3.Write a case study to identify and analyze the requirements of ARMS Portals, and prepare negative test scenarios as well as test cases for the application.

4.Write a case study to identify and analyze the requirements of E-Commerce Applications, and prepare positive test scenarios as well as test cases for the application.

5.Write a case study to identify and analyze the requirements of E-Commerce Applications, and prepare negative test scenarios as well as test cases for the application.

6.Write a case study to identify and analyze the requirements of Food Delivery Applications, and prepare positive test scenarios as well as test cases for the application.

7.Write a case study to identify and analyze the requirements of Food Delivery Applications, and prepare negative test scenarios as well as test cases for the application.

8.Write a case study to identify and analyze the requirements of Banking Applications, and prepare positive test scenarios as well as test cases for the application.

9.Write a case study to identify and analyze the requirements of Banking Applications, and prepare negative test scenarios as well as test cases for the application.

10.Develop a comprehensive Test Plan document for a chosen application (such as a Library Management System), outlining various testing strategies, methodologies, and criteria to ensure thorough evaluation and validation of the system's functionality.

11.Write a case study to identify and analyze the requirements of a Railway Reservation System and prepare positive test scenarios and test cases for the application.

12.Write a case study to identify and analyze the requirements of a Railway Reservation System and prepare negative test scenarios and test cases for the application.

13.Write a case study to identify and analyze the requirements of an Online Examination System and prepare positive test scenarios and test cases for the application.

14.Write a case study to identify and analyze the requirements of an Online Examination System and prepare negative test scenarios and test cases for the application.

15.Develop a comprehensive Test Plan document for an Inventory Management System, covering test scope, test design techniques, test execution strategy, and test completion criteria.

16.Demonstrate the working of JUnit to check whether a string is an anagram of another string and use assert statements to validate the output using white-box testing.

17.Write a white-box JUnit test case to find the maximum of three numbers and validate the output using appropriate assert methods.

18.Write a JUnit test case to check whether a number is even or odd, and validate the result using white-box testing techniques.

19.Write a program using functions to calculate compound interest.
Verify the output values using white-box testing and JUnit assertions.

20.Write a JUnit program to count the number of vowels and consonants in a given string and validate the output using assert statements.

21.Write a program to check whether a given number is a prime number.
Verify the output using white-box testing with JUnit assertions.

22.Write a JUnit test case to reverse a number and validate the output using white-box testing techniques.

23.Write a program to find the sum of digits of a given number and verify the output using white-box testing.

24.Write a JUnit program to check whether two strings are equal ignoring case sensitivity, and validate the output using assert statements.

25.Write a program using functions to calculate the grade of a student based on marks.
Verify the output values using white-box testing.

26.Write a JUnit test case to find the largest element in an array and validate the result using white-box testing techniques.

27.Write a program to calculate the average of n numbers using functions and verify the output using white-box testing.

28.Write a JUnit program to check whether a given number is an Armstrong number and validate the output using assert statements.

29.Write a program to convert temperature from Celsius to Fahrenheit.
Verify the output values using white-box testing.

30. Write a JUnit test case to find the length of a string without using built-in functions and validate the output using white-box testing techniques.
31.Demonstrate the working of JUnit to reverse a word and use assert statements to validate the output.

32.Write a white-box test case (JUnit) to perform string comparison and use assert statements to validate the output.

33.Write a JUnit test case for a voting system, use assert statements, and validate it using white-box testing techniques.

34.Write a program using functions to calculate Simple Interest.
Suppose the customer is a senior citizen.
He is offered 12 percent rate of interest;
for all other customers, the rate of interest is 10 percent.

The output values should be verified using white-box testing.

35.Check whether the given number is a palindrome or not, and verify the output values using white-box testing.

36.Write a Selenium automation script to open the Chrome browser and verify the title of ARMS portal
37.Write a Selenium automation script to automate search functionality in Google using the Chrome browser.

38.Write a Selenium program to automate user registration form filling
(enter name, email, password, and submit) using the Chrome browser.

39.Write a Selenium automation script to automate login into login into ARMS portal and validate error message for invalid credentials
40.Write a Selenium automation script to automate logout functionality from a web application using the Chrome browser.

41.Write a Selenium automation script to automate dropdown selection (using selectByVisibleText) on a web application using the Chrome browser.

42.Write a Selenium program to automate checkbox and radio button selection on a form using the Chrome browser.

43.Write a Selenium automation script to handle JavaScript alert, confirm, and prompt pop-ups using the Chrome browser.

44.Write a Selenium program to automate file upload functionality on a web application using the Chrome browser.

45. Write a Selenium automation script to capture a screenshot of a web page and store it in the local system using the Chrome browser.

46.Perform load testing on an e-commerce website during festival sale time by simulating 500 concurrent users accessing the homepage and analyze response time, throughput, and error rate using Apache JMeter.

47.Test a mobile banking REST API that handles balance enquiry requests by simulating multiple users and verify HTTP status codes, response data, and response time using Apache JMeter.

48.Perform POST request performance testing with JSON payload for a user registration service and evaluate system behavior for valid and invalid inputs under heavy load using Apache JMeter.

49.Test the login functionality of a university student portal where hundreds of students log in simultaneously to view exam results and analyze server stability using Apache JMeter.

50.Perform smoke and load testing on a newly deployed government service website to verify availability and performance of critical pages using Apache JMeter.

51.Conduct stress testing on a result-publishing server by gradually increasing the number of concurrent users to identify the system’s breaking point using Apache JMeter.

52.Analyze performance metrics (response time, throughput, error percentage) of a real-time online payment gateway under continuous transaction load using Apache JMeter.

53.Perform data-driven testing on a login API by reading thousands of user credentials from a CSV file and validating authentication responses using Apache JMeter.

54.Test session management and cookie handling of an online shopping cart application to ensure user sessions persist correctly during high-traffic navigation using Apache JMeter.

55.Perform end-to-end performance testing of a large-scale e-commerce application by simulating real user flow: login → browse products → add to cart → checkout → logout using Apache JMeter.
56.Perform capacity testing on an online movie ticket booking system by simulating a gradually increasing number of users during peak booking hours and analyze maximum supported users, response time degradation, and system stability using Apache JMeter.
57.Test the search functionality performance of an e-commerce website by simulating multiple concurrent users performing keyword searches and analyze average response time, throughput, and error percentage using Apache JMeter.
58.Perform endurance (soak) testing on an online learning management system (LMS) by running a constant load of users accessing video lectures for an extended duration and analyze memory leaks, response consistency, and error rates using Apache JMeter.
59.Test the file upload and download performance of a cloud storage web application by simulating concurrent users uploading and downloading files of varying sizes and analyze transfer rate, response time, and server resource utilization using Apache JMeter.
60.Perform API performance testing on a microservices-based weather information service by simulating high-frequency GET requests and analyze latency, throughput, and failure rate under peak load conditions using Apache JMeter.